QuickBite

Food Delivery Mobile App

Flutter • FastAPI • Riverpod • Isar

Type	Platform	Build Time	Screens
Portfolio Project	Android & iOS	3–5 Days	10 Screens

1. Project Overview

Purpose, goals, and what this project demonstrates

1.1 Purpose

QuickBite is a portfolio-grade food delivery mobile app built with Flutter and FastAPI. It is designed to demonstrate production-level cross-platform development skills to potential clients on Fiverr and other freelance platforms.

The app simulates a real-world food ordering experience — from browsing restaurants to tracking an order — using a clean UI, real API integration, and proper state management.

1.2 What This Demonstrates

Flutter Skills	Backend Skills
✓ Complex multi-screen navigation	✓ RESTful API design with FastAPI
✓ State management with Riverpod	✓ Pydantic v2 data validation
✓ Custom UI components & theming	✓ SQLite/JSON-based data layer
✓ Async API integration	✓ CORS, routing, response models
✓ Local data persistence (Isar)	✓ Modular project structure

2. Tech Stack

All tools, packages, and frameworks used

2.1 Flutter (Frontend)

Package	Version	Purpose
flutter_riverpod	^2.5.x	State management — providers, notifiers
go_router	^13.x	Declarative navigation & deep linking
dio	^5.x	HTTP client for API calls
isar	^3.x	Local database for cart & user data
cached_network_image	^3.x	Efficient image loading & caching
flutter_svg	^2.x	SVG icons and illustrations
shimmer	^3.x	Skeleton loading placeholders
gap	^3.x	Clean spacing in layouts
freezed	^2.x	Immutable data models & unions
json_serializable	^6.x	JSON ↔ Dart model generation

2.2 FastAPI (Backend)

Package	Version	Purpose
fastapi	^0.111.x	Core API framework
uvicorn	^0.30.x	ASGI server
pydantic	v2.x	Request/response validation
python-jose	^3.x	JWT token handling
passlib	^1.x	Password hashing (bcrypt)
sqlalchemy	^2.x	ORM for database models
aiosqlite	^0.20.x	Async SQLite driver

3. Architecture

Project structure and design patterns

3.1 Flutter Project Structure

lib/ — Project Layout

```
lib/
├── main.dart                    # App entry, providers, router setup
├── core/
│   ├── constants/               # Colors, strings, endpoints
│   ├── theme/                   # AppTheme, text styles
│   └── utils/                   # Helpers, formatters
├── data/
│   ├── models/                  # Freezed data models
│   ├── repositories/            # Data access layer
│   └── services/                # Dio API service, Isar service
├── providers/                   # Riverpod providers & notifiers
└── features/
    ├── home/                    # Restaurant listing screen
    ├── restaurant/              # Detail + menu screen
    ├── cart/                    # Cart & checkout screens
    ├── orders/                  # Order history & tracking
    └── profile/                 # User profile screen
```

3.2 FastAPI Project Structure

backend/ — Project Layout

```
backend/
├── main.py                      # FastAPI app instance, router includes
├── database.py                  # SQLAlchemy engine & session
├── models/                      # SQLAlchemy ORM models
│   ├── restaurant.py            # Restaurant & MenuItem models
│   └── order.py                 # Order & OrderItem models
├── schemas/                     # Pydantic v2 request/response schemas
├── routers/
│   ├── restaurants.py           # GET /restaurants, GET /restaurants/{id}
│   ├── menu.py                  # GET /restaurants/{id}/menu
│   └── orders.py                # POST /orders, GET /orders/{id}
└── seed.py                      # Seed script for test data
```

3.3 State Management Pattern (Riverpod)

The app uses Riverpod 2.x with the NotifierProvider pattern. Each feature has its own notifier class that manages business logic, keeping UI widgets completely stateless.

State Flow

```
UI Widget  →  reads Provider  →  Notifier  →  Repository  →  API / Isar
UI Widget  ←  rebuilds on state change  ←  Notifier emits new state
```

4. Screen Specifications

All 10 screens — layout, components, and data

#	Screen	Key Components	Data Source
1	Splash Screen	App logo, animated fade-in	—
2	Onboarding (3 slides)	Illustration, title, CTA button	Static
3	Home Feed	Search bar, category chips, restaurant cards, promo banner	GET /restaurants
4	Restaurant Detail	Header image, info row, tab bar (Menu / Reviews)	GET /restaurants/{id}
5	Menu Screen	Category sections, item cards, add-to-cart button	GET /menu/{restaurantId}
6	Cart	Item list, quantity stepper, price breakdown, checkout CTA	Isar (local)
7	Checkout	Address field, payment method selector, order summary	Isar + POST /orders
8	Order Confirmation	Success animation (Lottie), order ID, ETA	POST /orders response
9	Order Tracking	Status stepper, map placeholder, driver info card	GET /orders/{id}
10	Profile	Avatar, name, order history list, settings rows	Local + GET /orders

4.1 Home Feed — Detailed Breakdown

This is the most complex screen and the one that appears in portfolio screenshots first. It should be the most polished.

- **App Bar:** City selector (dropdown), cart icon with badge count
- **Search Bar:** Rounded container, search icon, placeholder text — tapping navigates to search screen

- **Category Chips:** Horizontal scroll, emoji + label, active chip highlighted in brand color
- **Promo Banner:** Full-width card with gradient overlay and promo text
- **Section Header:** "Nearby Restaurants" title + "See All" link
- **Restaurant Card:** Image (cached), restaurant name, rating stars, delivery time, delivery fee, cuisine tag
- **Loading State:** Shimmer skeleton matching card layout

5. API Endpoints

FastAPI backend contract

Method	Endpoint	Auth	Description
GET	/restaurants	None	List all restaurants with basic info
GET	/restaurants/{id}	None	Single restaurant detail
GET	/restaurants/{id}/menu	None	All menu items grouped by category
POST	/orders	Bearer JWT	Create new order from cart payload
GET	/orders/{id}	Bearer JWT	Get order status and details
GET	/orders	Bearer JWT	List all orders for current user
POST	/auth/register	None	Register new user
POST	/auth/login	None	Login, returns JWT token

6. Data Models

Dart (Freezed) and Python (Pydantic) schemas

6.1 Restaurant Model

Dart — Restaurant

```
@freezed
class Restaurant with _$Restaurant {
  final int id;
  final String name;
  final String imageUrl;
  final double rating;
  final int deliveryTimeMinutes;
  final double deliveryFee;
  final String cuisineType;
  final bool isOpen;
}
```

Python

```
class Re
    id:
    name
    imag
    rati
    deli
    deli
    cuis
    is_c

    mode

    )
```

6.2 Order Model

Dart — Order

```
@freezed
class Order with _$Order {
  final int id;
  final List<OrderItem> items;
  final double totalAmount;
  final OrderStatus status;
  final DateTime createdAt;
}

enum OrderStatus {
  pending, confirmed,
  preparing, onTheWay,
  delivered, cancelled
}
```

Python

```
class Or
    rest
    iten
    addr
    payn

class Or
    id:
    stat
    tota
    crea
    esti
```

7. Day-by-Day Build Plan

3–5 day execution schedule

Day	Focus	Tasks	Done When
Day 1	Foundation + Home	Project setup, routing, theming, splash, onboarding, home feed UI with shimmer loading	Home screen renders with dummy data
Day 2	Restaurant + Menu	Restaurant detail screen, menu list UI, category tabs, add-to-cart logic with Riverpod	Can add items to cart from menu
Day 3	Cart + Checkout	Cart screen with quantity controls, checkout screen, price breakdown, form inputs	Checkout screen submits cart data
Day 4	FastAPI Backend	FastAPI setup, SQLAlchemy models, seed data, all endpoints, JWT auth, connect Flutter	Flutter fetches real data from API
Day 5	Polish + Portfolio	Order confirmation animation, tracking screen, UI polish, screenshots, demo recording	Screenshots ready for Fiverr gig

8. UI / UX Guidelines

Colors, typography, spacing, and component rules

8.1 Color Palette

Primary Blue	Accent Orange	Dark Text	Background
#2563EB	#F97316	#111827	#F3F4F6

8.2 Typography

Style	Font	Size / Weight	Usage
Display	Poppins	24sp / Bold	Restaurant names, screen titles
Headline	Poppins	18sp / SemiBold	Section headers, card titles

Style	Font	Size / Weight	Usage
Body	Inter	14sp / Regular	Descriptions, menu items
Caption	Inter	12sp / Regular	Tags, ratings, metadata
Button	Poppins	16sp / SemiBold	CTA buttons, tab labels

8.3 Spacing & Layout Rules

- Use 8px base grid — all spacing values should be multiples of 8 (8, 16, 24, 32...)
- Screen horizontal padding: 16px on all screens
- Card border radius: 12px. Button border radius: 8px. Chips: 20px (pill)
- Card elevation: use BoxShadow, not Material elevation — more control
- Bottom navigation bar height: 64px with SafeArea padding
- All images should use BorderRadius.circular(12) with ClipRRect

9. Portfolio & Showcase Tips

How to present this on Fiverr

9.1 Screenshots to Capture

These are the 3 screenshots that should appear in your Fiverr gig image — in order of impact:

#	Screen	Why It Works
1	Home Feed	First impression — shows cards, images, real data. Clients immediately recognize this as a real app.
2	Restaurant Detail + Menu	Shows navigation depth and content-rich UI. Demonstrates you can handle complex layouts.
3	Cart or Checkout	Shows transactional UI — clients building e-commerce or food apps care about this most.

9.2 Demo Video Tips

A 30–60 second screen recording dramatically increases conversion on Fiverr. Record in this order:

- Open the app — show the splash and home screen loading with shimmer, then data appearing
- Tap a restaurant — show the smooth navigation transition and menu loading

- Add 2–3 items — show the cart badge incrementing in real time
- Go to cart — show the total updating and quantity controls working
- Place order — show the confirmation animation

Use a physical device if possible. Emulator recordings look less impressive.

9.3 GitHub Repo Checklist

Before sharing the repo link in your Fiverr gig, make sure:

- README.md has a GIF or screenshot at the top
- Setup instructions are clear (flutter pub get, backend setup, how to run)
- No API keys or secrets are committed (use .env files)
- Folder structure matches the architecture in this document
- At least 1 commit per feature — shows clean development process

Build it clean. Ship it fast. Let the code speak.

QuickBite — Flutter Portfolio Project | Musab